

VULVALIDATE Supplement: Protocol, Evidence, Review, and Experimental Details

ANONYMOUS AUTHOR(S)

S1 Operational protocol and agent configuration

This supplement provides the operational details supporting the main paper. Supplementary sections are numbered S1–S8 and are cited as “Section S_n ”; the bare labels S1–S6 denote the workflow stages defined in the main paper.

S1.1 Controller and unit of analysis

The audit used Claude Opus 5.0 [2] through Claude Code [1] with thinking effort = high for the workflow stages S1–S6. This configuration applies to historical reconstruction, experiment planning, attack construction and execution, evidence preservation, and adjudication for the originally vulnerable instances in BigVul [10], PrimeVul [9], and DiverseVul [6]. The later review of 581 instances is a separate activity described in Section S4.

The unit of adjudication is one dataset instance containing a labelled function. Project, fixing commit, CVE, and function identity provide context. Shared builds reduce repeated reconstruction work, but each instance still requires its own source binding, execution evidence, attribution, and outcome. The controller may use an existing experiment as a starting point only after checking that its source revisions and tested function match the new instance.

S1.2 Controller instructions

This subsection renders the operational contract in English, with one part for each workflow stage S1–S6 of the main paper’s method. It condenses the campaign instructions; the dataset-specific prompts and the reusable skill in the artifact (Section S8) retain the full rules and their revision history. Sections S2 and S3 define the requirements referenced in the stages below.

Input and output. The controller receives the dataset and instance identifier; the original label; the stored function bodies; the project, commit, file, function, CVE, and CWE metadata; the historical source; and any existing experiment records and attack history. It produces one audit outcome, the reason and scope of that outcome, and the preserved evidence.

S1.2.1 Step S1: Reconstruct the historical target and execution environment. The controller verifies the historical source and the direction of the listed repair, and binds the tested function to the complete upstream function. It preserves the stored bodies and records any mismatch or missing context.

S1.2.2 Step S2: Analyze the vulnerability mechanism and plan the experiment. The controller reads the fixing diff and the function’s callers and callees. The plan states the suspected mechanism, the required program state, the execution route, the appropriate oracle, the expected observation, the controls, and the stopping condition.

S1.2.3 Step S3: Agentic trigger construction. The controller constructs or recovers an attack input or runtime condition. Any harness preserves the relevant guards, types, and calling contracts. The controller executes the attack and uses coverage and oracle feedback to refine it, appending every material attempt, including failures, to the attack history. Section S2.3 details the steps of this search.

S1.2.4 Step S4: Controlled two-side validation and function-level attribution. The controller replays the retained condition on the vulnerable and fixed source sides, holding the driver, input, environment, limits, and oracle settings fixed where feasible. Deviations are recorded, and the comparison is narrowed accordingly. This stage establishes target execution, exercise of the mechanism, and attribution of the fault.

S1.2.5 Step S5: Evidence preservation, Docker packaging, and cold replay. The controller preserves source and build bindings, raw inputs, complete logs, return codes, execution proof, replay commands, and the evidence map. Container-compatible experiments are packaged with their environment and Dockerfile [22], together with the base-image pin and the case image identity. A cold replay is attempted after the identified generated products are removed, in a fresh container isolated from the original workspace. New replay output is stored separately, and the integrity-check results are recorded.

S1.2.6 Step S6: Evidence-based interpretation and adjudication. The controller confirms a label when a faithful, target-attributed defect witness is obtained. It corrects a label only with a supported rejection of the original attribution and the applicable dynamic evidence, including all seven negative checks. A usable but inconclusive attack is retained as attacked but undecided, and the measurement-failure outcome is assigned when usable observations remain unavailable. The record preserves the outcome and its provenance.

Requirements across stages. Observations are reported from saved files. Each decision requires its stated oracle, source binding, and attribution. Manual review is separate from S1–S6; for reviewed instances, only those with a final Supported consensus verdict are retained in either learning class.

The controller stops an individual search when it has evidence sufficient for adjudication or reaches a documented technical or resource limit. Reaching a limit preserves the observation and its unresolved questions. Completion of an attempt, acceptance of a decision, and successful replay are reported separately.

S2 Dynamic tools, oracles, and execution routes

S2.1 Mechanism-specific observations

An oracle specifies which runtime observation would evidence the suspected defect. Tool selection follows the fixing diff and source-level mechanism, with the recorded CWE used as context. Table S1 summarizes the tool portfolio. A tool’s silence is informative only for mechanisms that the configured tool can observe.

Equal return codes can accompany a real behavioral difference, such as different authorization decisions or disclosed output. Conversely, a nonzero code can result from a missing dependency or a harness failure. The record therefore includes the actual oracle observation as well as the exit status. A sanitizer report must be mapped to a qualified function and source-line range. Reports from a caller, a sibling function, or the harness require a demonstrated causal link to the labelled function.

When the sensitivity of an oracle is in question, a retained control establishes that it can detect the proposed effect: a positive control must produce the expected signal in a retained execution, and a benign input separates a selective failure from an environment that fails on every run.

Tool-selection and reading rules. The following rules turn Table S1 into concrete decisions. Rules 3–8 originate in failure modes observed during the campaign.

- (1) *Mechanism first.* The oracle follows the mechanism read from the fixing diff: ASan for out-of-bounds access and use-after-free, UBSan for integer and representation errors, MSan or

Table S1. Tools and observations used to investigate different mechanisms. Coverage and debugger traces establish execution and attribution; the vulnerability observation comes from a mechanism-appropriate oracle.

Mechanism	Tools or observation method	Evidence to retain
Invalid memory access	ASan, KASAN	Invalid access, affected object, and allocation/free and fault traces, as applicable [17, 25].
Uninitialized data	MSan, KMSAN, Memcheck	Use of uninitialized data, origin information when available, and its relation to the observed behavior [19, 23, 29].
Arithmetic or representation	UBSan, width-specific builds, value checks	The reported operation and operands; use a matching integer width when the mechanism depends on it [21].
Leaks and lifetime loss	LSan, Memcheck	Unreleased allocation and its lifetime or resource effect, excluding harness-owned allocations [20, 23].
Concurrency	TSan, KCSAN, controlled schedules	Conflicting accesses or a repeatable schedule linked to the target mechanism [18, 26].
Denial of service	Time, progress, signal, and resource observations	The stopping bound, observed progress or resource use, and behavior under the same limits on the comparison side.
Authorization, injection, or state errors	Decision, output, or state checks	The security-relevant decision or effect and the expected behavior under the same input [3].
Execution and attribution	Coverage, debugger, and stack traces	Target execution, disputed-line or branch coverage, and the relevant call and data relationships [15].

Memcheck for uninitialized reads and information disclosure, time and resource bounds for denial of service, and decision or output differentials for authorization, authentication, and injection. ASan instruments addressability only, so uninitialized-value and disclosure mechanisms require MSan or Memcheck.

- (2) *Matching width*. Integer mechanisms that depend on a 32-bit `size_t` or pointer width are compiled with `-m32`.
- (3) *Report prefixes*. ASan and LSan reports use the prefix `ERROR::`; MSan and TSan reports use `WARNING::`; and a UBSan diagnostic always begins with `<file>:<line>:<column>: runtime error: .` Report parsers match each tool's own prefix and source location, which separates sanitizer findings from diagnostic text printed by the program itself.
- (4) *Upstream invocation*. Command-line options follow the project's own regression tests. In two `tcpdump` instances (BigVul 183013 and 181110), a verbosity flag added by a script bypassed the branch containing the check; with the upstream test options, both instances fault in the labelled function on the vulnerable side and run cleanly on the fixed side.
- (5) *Complete logs*. Every fault in a log is attributed, including faults after the first. In one Android decoder instance (BigVul 188078), the first fault of every log lies in an API wrapper, and a later fault inside the labelled function appears in 72 of 78 logs.

- (6) *Line-range attribution.* Stack frames are matched by qualified function identity and source-line range. Overloads and similarly named functions make bare-name matching unreliable: in BigVul 187856 and seven sibling instances, the labelled `EBMLHeader::Parse` shares its bare name with the faulting `mkvparser::Block::Parse`, one of 11 `Parse` overloads in that parser. C++ frame names may also contain spaces. A body file can serve as the attribution key once its content is shown to hold the labelled function alone; in BigVul 180472 the body file holds 41 functions, and the faults belong to `print_object` rather than the labelled `ipow`.
- (7) *Unattributed output.* Sanitizer reports whose stacks exclude the labelled function are recorded as unattributed output with their count, signature, and an example. In BigVul 181344, for example, the reports stem from the driver's own `calloc`. Such output leaves the interpretation open in both directions.
- (8) *Execution evidence.* A debugger (gdb [12]) hit count of zero for a static function compiled with optimization is cross-checked with a second method, such as an inlining-aware sanitizer frame, gcov [14], or a function-specific output string, because inlining can leave the breakpoint on an unused out-of-line copy. This occurred for `rx_cache_find` in BigVul 183017, settled by an ASan frame, and for `parse_wcc_attr` in BigVul 181112, settled by its unique output string.
- (9) *Execution hygiene.* Each run uses a timeout, disables core dumps, and clears `LD_LIBRARY_PATH`, so both sides load the libraries built for them.

S2.2 Route selection and controlled comparison

The preferred full-project route rebuilds the historical project and invokes its public parser, decoder, service, command, or API with a saved input. Existing regression inputs and pinned builds may be reused after checking their revision and function identities. This route retains the program context, but a fixing commit that changes several functions requires additional evidence for function-level causality.

When the full-project route is infeasible, the extracted-function route compiles the actual function body with the required types, guards, state, and callees. The driver supplies a calling context permitted by the original program. A local comparison can isolate the function change more closely, but its interpretation depends on the fidelity of the reconstructed context. Removing a pre-existing guard or hand-writing the intended defect into the driver invalidates that comparison.

Containers reproduce historical user-space dependencies. Kernel experiments may use a faithful local harness or a version-compatible kernel under QEMU/KVM [4, 16] with appropriate instrumentation. Android and media experiments may reuse a pinned decoder or extracted calling context. An unavailable platform or dependency is recorded with the attempted route and actual failure log.

Both executions retain the same input, driver, compiler and relevant flags, dependencies, instrumentation, environment, and resource limits wherever the comparison permits. Source and binary hashes identify what was tested. Identical binaries require an explanation before a claimed repair difference is accepted. A structural inability to construct one side, an interface mismatch, or a code-generation equivalence is recorded as a comparison limitation, with the actual available execution preserved.

S2.3 Feedback loop and stopping conditions

The attack search of stage S3 follows the six steps below; they detail S3 and are distinct from the workflow stages S1–S6. Each refinement retains the previous input and observation, allowing a reader to trace how the final experiment was selected.

- (1) *Specify the mechanism.* Read the historical source, repair, and calling context; identify the disputed operation, required state, candidate entry point, and expected security effect.

- (2) *Select an execution route and oracle.* Choose a historical project build or faithful harness and configure an observation method that can detect the proposed effect. Record applicable controls.
- (3) *Construct and execute the attack.* Save the input, schedule, or controlled runtime condition, then retain complete output and actual return codes from each executed source side.
- (4) *Inspect execution and attribution.* Read the observations at the three levels used in the main paper: function entry establishes that the target was reached, line or branch coverage establishes execution of the disputed operation, and the oracle establishes a security-relevant violation. Check source identity and the call or data path connecting the target to the observed effect.
- (5) *Refine from feedback.* The first unmet level selects the next change. An unreached function revises the input or entry path; an unexecuted disputed operation revises the triggering condition, harness, or program state; an inadequate observation revises the tool, oracle, or control; a reconstruction failure revises the build route. Preserve the reason for each change.
- (6) *Freeze, preserve, and adjudicate.* When a trigger succeeds, freeze the exact input, harness, source and environment bindings, commands, logs, and coverage for two-side replay in S4. Bind the retained observations to their replay package and apply the S6 criteria in Section S3.

Stopping conditions. An individual search ends at the first of the following conditions, and the stopping record names which one applied.

- (a) *Witness obtained.* A target-attributed violation is observed on the vulnerable side and replayed on both sides under the frozen condition.
- (b) *Negative contract met.* An attack that exercises the condition named by a source-grounded rejection rationale R satisfies all seven checks of the main paper's Table 1.
- (c) *Technical limit.* Source, dependency, toolchain, or platform recovery fails on every attempted route, and the build logs of those routes are retained.
- (d) *Resource limit.* The documented time, iteration, or resource bound is reached; symmetric timeouts on both sides are recorded as resource observations.

Conditions (a) and (b) lead to *Confirmed* and *Label correction*, respectively; conditions (c) and (d) lead to one of the two unresolved outcomes, according to whether a usable dynamic observation exists. The stopping record identifies the exercised attack surface, attempted refinements, applicable limits, and remaining evidence needs. A clean execution contributes a bounded observation, and label correction requires condition (b) in full.

S3 Evidence fields, attack history, adjudication, and replay

S3.1 Evidence records and package layout

Table S2 specifies the information needed to interpret an experiment. Unmeasured or unknown observations carry explicit availability markers, and a zero or a passed check is recorded only when it was measured.

For each executed side, the record stores the return code and the observed output. The return codes are integers produced by actual executions; textual placeholders such as "identical" or "n/a" fail the record checks. A record can report, as a measured result, that the labelled function remained unexecuted.

Each sample's evidence package contains the replay script, driver or harness, tested bodies or source pins, raw inputs, complete logs, execution proof, and evidence map. The map associates each active file with its relative path, purpose, size, and SHA-256 digest.

Shared historical builds reside in a build bank within the repository. Their records identify the upstream remote, precise revisions, compiler, configuration, dependencies, local changes, and

Table S2. Evidence fields and their interpretation. Paths identify retained files relative to the sample or repository, according to the record schema.

Field group	Required information
Identity and provenance	Dataset and sample ID, original label, project, function/file identity, source revisions, and body hashes.
Mechanism and input	The root cause, the trigger mechanism, the attack inputs, and the input's construction intent.
Method and oracle	The dynamic method and route, the oracle and the reason it was chosen, commands, and configuration.
Observed comparison	The two-sided comparison, with actual integer return codes and separately identified observations and full logs for each executed side.
Execution and attribution	Whether the labelled function executed and the proof of execution, disputed-line coverage, qualified function identity, and relevant call/data path.
Trigger scope	Whether the trigger is synthetic or attacker controlled, and the evidence for a faithful runtime condition.
Negative interpretation	The basis of the label correction, the seven-check evidence, and the source-grounded rejection rationale.
Search and persistence	The tested attack surface, attempt history, stopping reason, replay script, and EVIDENCE_MAP.json.

any saved patch. Each sample identifies the shared build it used and preserves its own input and observations.

S3.2 Docker packaging and execution boundaries

Container-compatible experiments use one case package per instance. The package binds the stored function bodies, harness, inputs, attack history, and evidence map to a Dockerfile and an execution environment. The vulnerable and fixed builds remain separately identifiable. A shared build is included through a documented build recipe when the experiment depends on it. Each dependency receives an explicit reconstruction recipe.

The case-image builder for confirmed instances copies this material into its build context and invokes the saved replay script inside the image. Its base Dockerfile pins Ubuntu 22.04 by image digest and installs the compilation, debugging, and scripting tools needed by the case recipes. Additional dependencies and upstream libraries are specified by those recipes. The base-image digest fixes the starting environment. Packages obtained subsequently have their own version and provenance requirements. Exact dependency versions, source revisions, and build settings therefore remain part of the reconstruction record.

Replay uses a fresh container with networking disabled and the original workspace isolated. The runner captures the complete container output and integer exit status in a separate replay record. A successful container launch and script completion establish execution; reproducing the vulnerability additionally requires the relevant oracle output, source-side identity, and function-level attribution. Resource limits and sanitizer settings must match the intended mechanism; for example, a memory-leak experiment requires enabled leak reporting and a working leak-detection control.

The release index described in the main paper links each instance and audit outcome to its image identity and replay command. The implemented build record preserves the image tag, base-image

reference, dependencies, source provenance, and any captured run log and exit status. For the release index to support reconstruction claims, its records must also distinguish image construction, execution, clean rebuilding, and oracle reproduction. The image digest provides the immutable executable identity; the tag provides its human-readable reference. Experiments requiring a particular kernel, virtual machine, device, or privilege retain those requirements explicitly.

S3.3 Attack history and persistence

Each material attempt records its order or timestamp; hypothesis and entry point; source/build identity; saved input, schedule, or injected condition; commands and oracle settings; return codes and full output; achieved target and disputed-line coverage; attribution; limiting factor; and the reason for the next refinement or for stopping. The history can span a local harness, a full project build, and more than one oracle.

Failed attempts remain alongside successful ones, including earlier synthetic inputs, missing dependencies, uncovered branches, and unrelated faults. When an evidence file is superseded, its earlier version and role remain identifiable; the active evidence map points to the files supporting the current decision. Changed hashes are investigated before the map is refreshed.

S3.4 Outcome rules

Confirmed. A complete, source-bound function must execute under a faithful input or runtime condition and produce a real, target-attributed defect observation.

Controlled failure injection. Some mechanisms depend on a runtime condition that an input file alone leaves unreachable, such as an allocation failure, an interrupted system call, or a short read. An injected condition supports confirmation only when all five of the following hold: (i) the upstream repair explicitly addresses the injected condition; (ii) the injection uses a public or upstream-provided interface and leaves upstream code unchanged; (iii) the same injector runs on both sides; (iv) the evidence links the condition to the fixing diff and shows that the condition occurred and reached the disputed operation; and (v) a positive control demonstrates that the injector and oracle work. Attacker control of the condition is a separate claim with its own entry-path evidence.

Three trigger forms remain disqualifying for confirmation: synthetic values outside the range that the real entry accepts or produces, such as arbitrary magic constants; defects created by the harness, for example by compiling out an upstream guard; and a vulnerable side that stays clean while only the comparison side faults.

Label correction. For a mechanism-specific dynamic negative, the seven checks in the main paper's Table 1 are necessary: measured execution, function execution, attack input, oracle relevance, search scope, source completeness, and patch execution. Acceptance also requires a source-grounded rationale explaining why the exercised behavior rejects the original attribution. The attack must exercise the condition addressed by that rationale. Let $C_1, \dots, C_7$ denote these checks and R the evidence-supported rejection rationale. The acceptance condition is

$$N_{\text{corr}} = \left(\bigwedge_{j=1}^7 C_j \right) \wedge R.$$

Section S5 gives a case (BigVul 187491) in which an existing retry already handles the challenged condition.

An apparent inverted differential, with a defect observed only on the comparison side, requires verification of source orientation, source and build bindings, and the observed defect, and still needs the rejection rationale R ; otherwise the instance remains *Attacked but undecided*, because

the comparison-side witness concerns only that side’s code. Static reading nominates a suspected correction; acceptance requires the dynamic evidence and rejection rationale together.

Attacked but undecided. A usable observation was obtained, and the evidence stays below the thresholds for both confirmation and exclusion. A documented single-side observation retains its comparison limitation. Each two-sided claim requires observations from both actual executions.

Not successfully measured. Usable dynamic observations remain unavailable. Build logs establish the reconstruction attempt and its outcome. Both unresolved outcomes are omitted from the two learning classes.

S3.5 Cold replay, integrity checks, and stopping

A cold replay first identifies and removes generated sample-specific products, then executes the saved reconstruction and replay procedure from retained source and input. For containerized experiments, this takes place in a fresh container isolated from the original workspace. It records rebuilt binary identities, actual return codes, complete observations, and evidence-map checks. Reusing an existing binary establishes binary replay. Source reconstruction requires a separate successful rebuild. Shared build pins remain explicit.

Some case recipes build an upstream library or the tested executables during image construction. Starting a fresh container from such an image demonstrates isolation from the original workspace. Cold reconstruction of prebuilt components requires a record identifying which products were removed and which source and dependency steps were rerun. Original evidence is retained unchanged, and new logs and hash checks are stored separately for comparison.

The repository’s sample verifier, prompt-compliance checkers, and evidence-location checker inspect required fields, paths, source bindings, hashes, and return-code consistency. Both compliance checkers run before a batch is closed: a short one for the most frequent violations and a full one that covers every machine-checkable rule of the protocol. These checks establish structural consistency. Harness fidelity and defect attribution additionally require source and execution interpretation.

An attempt can end with a documented unresolved outcome. A reproducibility claim additionally needs a successful replay within its stated environment, while acceptance of an adjudication requires its semantic evidence. Section S8 distinguishes these requirements from the completeness actually established for the retained artifact.

S4 Manual-review assessment

This section records how the verdicts of the main paper’s manual review were assigned and used. The selection of the 581 instances, the blinded two-expert procedure, and the checklist are specified in the main paper’s manual-review subsection and are not repeated here.

S4.1 Assessment and interpretation

The unit of review is a dataset instance, identified by its dataset and sample identifier. Repeated entries in an assessment history are revisions of one assessment and contribute only once.

Each verdict compares the final expert consensus with the initial agent decision. *Supported* means that the consensus supports that decision within its stated scope. *Contradicted* means that the evidence conflicts with it. *Insufficient evidence* means that a necessary part of its justification remains missing or unresolved. The verdict therefore assesses the initial decision, not the function directly, and it governs learning-set eligibility: a Supported confirmation remains eligible for the vulnerable class and a Supported label correction for the non-vulnerable class.

Of the 581 reviewed instances, the 545 with a Supported verdict pass the manual-review filter. The other 36, comprising 12 Contradicted and 24 Insufficient-evidence verdicts, are excluded from

both classes and grouped with *Attacked but undecided* in the post-review outcome distribution. Because release membership is resolved by body identity and provenance, these instance-level verdicts enter dataset construction through their linked records. Each review record links the initial S6 decision, the two individual assessments, the final consensus, and the post-review status to the frozen instance identifier.

S4.2 Comparison with independent manual labels

This section gives the full procedure and results behind the main paper’s comparison with two independent manual labellings.

Sources. Risse et al. [24] relabelled 100 BigVul and 100 DiverseVul functions that the datasets mark as vulnerable and answered, for each, whether the function is vulnerable. A negative answer carries one of three reason codes: code 1, the wrong fixing commit was identified; code 2, the change is structural; code 3, the change is unrelated to the vulnerability. For each function judged vulnerable they also recorded whether that judgment could be made from the function body alone and identified any required context: arguments, external functions, type declarations, globals or macros, or the execution environment. Croft et al. [8] manually checked 70 BigVul functions labelled vulnerable and answered whether the dataset’s label is accurate, assigning inaccurate labels to the categories cleanup, inaccurate, and irrelevant. All 70 of Croft et al.’s functions are among Risse et al.’s 100. Both studies’ released label files were used unchanged; Risse et al.’s files were fetched through the Git LFS batch API and each file’s SHA-256 was checked against its pointer before use, and Croft et al.’s archive is kept with its checksum.¹

Joining. Both studies identify a BigVul function by its row number in the released table, which is also the instance identifier in this work, so the join is exact. DiverseVul functions are joined on their index and a hash of the function text. Every externally labelled function joins to one audited instance: 100 of 100, 70 of 70, and 100 of 100. The comparison script requires a complete join before computing agreement.

From outcomes to decisions. Each external study gives a binary answer. On our side, *Confirmed* accepts the vulnerable label and *Label correction* rejects it. Agreement is evaluated on instances with a binary decision from both sides; instances with either unresolved outcome are not compared.

Each agent decision is read from the campaign measurement ledger, which records for every instance its outcome and the provenance of that outcome (a measurement made in this campaign or a decision imported from the instance record). The frozen comparison in the main paper reads every joined instance from this single representation.

Attribution of a confirmation. The S6 rule requires a real security violation causally attributed to the labelled function. A call path establishes reachability, not attribution. A callee-side violation supports confirmation only when the evidence identifies a mechanism-relevant action of the labelled function and links that action to the affected object or state and the downstream violation. Risse et al.’s explanations often focus on the defect addressed by the listed repair, while Croft et al. assess the original label and its commit attribution. These differences help interpret the disagreements below.

Measures. For each comparison we report n , the number of functions with a decision on both sides; raw agreement p_o ; the agreement expected by chance p_e , computed from the two sides’ marginal rates; and Cohen’s $\kappa = (p_o - p_e)/(1 - p_e)$ [7]. The same measures are computed between

¹<https://github.com/niklasrisse/TopScoreWrongExam>; https://figshare.com/articles/software/Reproduction_Package_for_Data_Quality_for_Software_Vulnerability_Datasets_/20499924.

Risse et al. and Croft et al. on their 70 shared functions as a reference for how much two human teams agree.

The joined label files, ledger entries, and instance-level reasons accompany the comparison. The human–human reference uses the same 70 shared BigVul functions independently of the agent’s decided subset.

Cells. Against Risse et al.: 31 functions both call vulnerable, 40 both reject, 17 the agent confirms and Risse et al. call secure, 4 the agent rejects and Risse et al. call vulnerable; $p_o = 71/92 = 0.772$, $p_e = 0.495$, $\kappa = 0.55$. Against Croft et al.: 23 both accept, 20 both reject, 9 the agent confirms and Croft et al. call inaccurate, 11 the agent rejects and Croft et al. call accurate; $p_o = 43/63 = 0.683$, $p_e = 0.501$, $\kappa = 0.36$. Risse et al. against Croft et al.: 54 of 70 agree, $p_e = 0.489$, $\kappa = 0.55$. On DiverseVul, against Risse et al.: 49 both vulnerable, 19 both reject, 10 the agent confirms and Risse et al. call secure, 9 the reverse; $p_o = 68/87 = 0.782$, $p_e = 0.559$, $\kappa = 0.50$. All three efforts reach the same decision on 37 of the 63 BigVul functions on which all three decided (58.7%).

Disagreements with Risse et al. Seventeen of the 21 are functions the agent confirms and Risse et al. call secure. Risse et al.’s reason codes for them are code 2 (structural changes) for 177987, 179119, 181056, 181865, 183100, 183552, 184382, 185183, 186995, and 187292; code 3 (unrelated changes) for 180879, 183240, and 187837; and code 1 (errors during patch commit identification) for 178636, 185913, 186231, and 186879. The grouping that follows uses the stated explanations rather than the reason codes. For seven of them—179119, 181056, 181865, 183100, 183240, 186995, and 187292—Risse et al. attribute the reported CVE’s defect to another function in the same commit. For the other ten, they judge that the labelled function’s change is not part of a security fix. Four state that the commit does not address a security vulnerability (178636, 185913, 186231, 186879). Six describe the change to the labelled function as incidental to the repair: a pure reindentation (187837), a move between files (183552), a callee signature change (185183), a callee rename (180879), added cleanup (184382), and a change to IPv4 handling where the listed defect concerns IPv6 (177987). One example illustrates the first group. BigVul 181056 (tcpdump, CVE-2017-13010): Risse et al. write that the over-read is in `l_strnstart` and that `beep_print` “is changed to adhere to the structural changes”. The agent’s record identifies the action of `beep_print` at line 44, which passes the packet buffer and its unchecked length to `l_strnstart`, and links it to the AddressSanitizer over-read of that buffer in the callee. This is the same observation scored under two different questions: whether an action of the labelled function causes a real violation, possibly in a callee, and whether the function contains the filed CVE’s defect. The other ten are a different divergence: there the two sides differ in their commit attribution, while the agent’s confirmation records an executed defect in the labelled function. The four disagreements in the other direction are 180692 (below), 183161, 186569, and 184584.

Disagreements with Croft et al. Eleven of the 20 are functions the agent rejects and Croft et al. judge accurately labelled. Ten of them—178219, 178949, 178986, 180255, 183188, 184397, 184709, 185822, 187332, and 187629—are also rejected by Risse et al. and appear in Table S4 as disagreements between the two human studies; on these ten the agent agrees with Risse et al. Each of the ten carries a recorded commit-level cause, listed in Table S3. The eleventh, 180692, is called vulnerable by both studies and is discussed below. All eleven carry a replay script, both sides’ logs, and execution evidence in their instance directories. Croft et al. rejected 32 of their 70 labels as cleanup, inaccurate, or irrelevant and judged these eleven accurate.

BigVul 183161 is an agreement with Croft et al., who classify it as cleanup, and a disagreement with Risse et al. The remaining nine disagreements—178636, 179119, 180879, 183240, 183552, 184382,

Table S3. The agent’s recorded reason for each of the ten label corrections that Croft et al. judge accurate and Risse et al. reject. Each reason names the commit-level cause and, where recorded, the location of the repaired defect.

ID	Project	Agent’s recorded reason
178219	PHP	The commit’s only semantic hunk is in <code>_xml_characterDataHandler</code> ; the labelled <code>xml_parser_dtor</code> receives only trailing-whitespace changes.
178949	Linux	Part of a commit-wide perf interface change; the defect lies in the sibling <code>__perf_event_overflow</code> path in <code>kernel/events/core.c</code> , not in <code>ptrace_hbptriggered</code> .
178986	Linux	Same commit-wide perf interface change; the defect lies in <code>__perf_event_overflow</code> , not in <code>do_mathemu</code> .
180255	PHP	The commit changes the target only in whitespace and indentation; the defect is the missing <code>length > INT_MAX</code> guard in <code>SplFileObject::fread</code> , which faults before the fix and is clean after it.
183188	ImageMagick	The commit’s semantic change replaces the blend-range read loop in <code>ReadPSDLayersInternal</code> ; the labelled <code>ReadPSDChannel</code> has no semantic change.
184397	Chrome	A mechanical rename from pointer-address strings to integer identifiers relabels all 20 touched print-preview functions; the disclosed value is produced by the caller <code>PrintPreviewUI::GetPrintPreviewUIAddress</code> .
184709	Chrome	The target only adds a member initializer supporting the WebUI-bindings repair; the access check lives in a sibling function (BigVul 184711).
185822	Chrome	The target only threads the isolate through <code>performCheckpoint</code> ; the defect lies in the microtask-observer registration in <code>WorkerThread</code> and <code>Microtask</code> .
187332	Chrome	A third-party libxslt roll bundles many upstream commits; the out-of-bounds defect lies only in <code>xsltNumberFormatGetMultipleLevel</code> in <code>numbers.c</code> , confirmed in BigVul 187331.
187629	Android	A whole-library libpng update (1.6.10 to 1.6.20, 207 files); the target lives in <code>contrib/</code> test code that the platform build does not compile, and the mechanism is in <code>png_handle_PLTE</code> .

185913, 186879, and 187292—are confirmations that Croft et al. call inaccurate, and every one of them is also among the disagreements with Risse et al.

The function both studies contradict. BigVul 180692 (krb5, CVE-2015-8631, `modify_policy_2_svc`) is called vulnerable by Risse et al., accurately labelled by Croft et al., and rejected by the agent. Risse et al.’s explanation is that `client_name` and `server_name` have incomplete initialization or release paths that can cause memory exhaustion, according to the patch message and CVE report. According to the agent’s record, the fixing commit addresses memory leaks in the `kadmin` server stubs, so the oracle is `LeakSanitizer` [20] with `detect_leaks=1` and `fast_unwind_on_malloc=0`; the second option was required because the default unwinder stopped inside the uninstrumented system GSSAPI library. A positive control was built and run first: a driver that allocates the same two GSS display buffers through the same library and retains both allocations, for which `LeakSanitizer` reports 55 bytes in two allocations with a full stack. Six crafted RPC requests then covered every early-exit path of the stub on both revisions, each exiting 0 with clean leak-oracle output; `gdb` breakpoints on the patched lines recorded hits, so the disputed code executed. The recorded rejection rationale is that the leaked object is created on the path `setup_gss_names` → `krb5_unparse_name(princ)`, which exists only in the stubs that take a principal argument; `modify_policy_2_svc` takes a policy

Table S4. All human–human disagreements on BigVul. Risse’s column gives the released explanation; Croft’s column gives label accuracy or its released rejection category.

ID	Risse’s assessment	Croft’s assessment
177987	Target handles IPv4; the listed defect concerns IPv6 MTU handling.	Accurate
178219	Whitespace change; defect is in <code>xml_parse_into_struct</code> .	Accurate
178949	Removal of an unused parameter.	Accurate
178986	Adaptation to the changed <code>perf_sw_event</code> signature.	Accurate
180255	Integer-overflow repair concerns another function.	Accurate
181056	Over-read originates in <code>l_strnstart</code> .	Accurate
181865	Integer overflow originates in <code>jas_image_create</code> .	Accurate
183161	Migration to <code>mbcache2</code> addresses denial of service.	Cleanup
183188	Indentation change; defect is in <code>ReadPSDLayersInternal</code> .	Accurate
184397	Target participates in the broader WebUI pointer-to-ID refactor.	Accurate
184584	Function is the target of the listed vulnerability repair.	Inaccurate
184709	Target initializes a flag introduced by the process-isolation repair.	Accurate
185822	Target adapts to the changed microtask interface.	Accurate
186995	Type confusion originates in <code>xmlXPathNextPrecedingSibling</code> .	Accurate
187332	Security repair concerns <code>numbers.c</code> ; target adapts template handling.	Accurate
187629	Target adapts to the changed <code>pixel_copy</code> interface.	Accurate

name, and the commit changed every stub in the file. Both human explanations reason from the commit message and the CVE text, whereas the agent’s rejection rests on execution. We retain this case as a documented disagreement between the two attribution judgments.

Disagreements between the human studies. Table S4 identifies all 16 differences in the 70-function human–human reference. Fourteen concern an accepted Croft label and a rejected Risse label; two have the reverse direction. The comparison retains each study’s original question and reason.

Disagreements on DiverseVul. Table S5 lists all 19 disagreements: ten confirmations against a rejected human label and nine corrections against an accepted human label. The table reports the reasons each side recorded, including the agent’s decisive observation. Detailed inputs and observations are linked through the joined table and measurement ledger.

Context dependence. Risse et al. record that all 39 functions they judge vulnerable in BigVul and all 65 in DiverseVul require context beyond the isolated function body. Their BigVul breakdown identifies 19 cases requiring external functions, 16 requiring arguments, three requiring globals or macros, and one requiring type declarations. Source context is therefore central to interpreting both agreements and disagreements.

Interpretation. The comparison measures agreement between evidence-based audit decisions and independent manual labels. The human teams disagree on 16 of their 70 shared functions, providing a reference for differences in judgment and attribution scope. The sampled population consists of originally vulnerable functions. At these sample sizes, one changed decision moves agreement by about one percentage point. Further measurement can change the decided subset and its agreement. The artifact preserves the frozen comparison, source checksums, joined identities, and both sides’ reasons for every disagreement.

Table S5. All DiverseVul disagreements in the main-paper comparison. IDs are the original DiverseVul indices; the joined artifact retains their full sample identifiers. R denotes Risse et al. and A the agent.

ID	Project	R's recorded reasoning	A's recorded decision
4807	JasPer	Security repair concerns other functions; target changes address naming.	Confirmed: a crafted JP2 file triggers UBSan signed overflow via jp2_decode; fixed side silent.
9674	DPDK	Resource exhaustion originates in vhost_user_get_inflight_fd.	Confirmed: a valid descriptor is overwritten without close; the fixed side closes it.
13276	njs	Repair concerns njs_await_fulfilled; target adapts to it.	Confirmed: upstream regression script triggers ASan use-after-free; fixed side clean.
2684	Linux	Target adapts to changes in xfs_da_args.	Confirmed: the call overwrites the caller's value length (64 to 65,536); fixed side preserves it.
878	Linux	Race is attributed to the page-table helper changed by the repair.	Confirmed: live huge-page entries are cleared as bad (849,867 of 2M walks); none when fixed.
7375	Amanda	Target adapts to the changed check_exec_for_suid interface.	Confirmed: the pre-fix client runs an attacker-named program as root; the fixed one refuses.
4977	QEMU	Target adapts to the move from CharDriverState to CharBackend.	Confirmed: front-end teardown triggers ASan use-after-free; fixed side clean.
2360	Linux	Denial of service originates in sctp_verify_asconf.	Confirmed: a crafted ASCONF chunk causes an ASan out-of-bounds read in the body.
13610	Linux	Repair concerns parse_rock_ridge_inode_internal.	Confirmed: chained CL records cause unbounded recursion and ASan stack overflow; fixed depth 2.
14908	OpenDoas	Environment-inheritance defect originates in createenv.	Confirmed: six attacker environment values reach execvpe; none when fixed.
16744	qpdf	Timestamp conversion may overflow with 32-bit time_t.	Label correction: project-wide sign-conversion sweep; the attacked clock read shows no fault.
16944	Samba	Commit and CVE describe a potential use-after-free.	Label correction: the mapped attribute is skipped one function earlier and never reaches the target.
9345	libyang	Schema lookup may cause an invalid memory access.	Label correction: the commit passes the value already used here; hostile arguments give identical output.
326	udev	NETLINK sender validation is implicated in privilege escalation.	Label correction: the credential check is in the uncalled sibling udev_monitor_receive_device.
7369	Amanda	Command-line validation is implicated in privilege escalation.	Label correction: the closed exploit path runs through six sibling estimate and backup functions.
14558	FFmpeg	Negative item_num may bypass validation.	Label correction: a full item_num sweep finds no fault; av_malloc rejects negatives.
17653	libsvg	Access to stop->rgba may dereference a null pointer.	Label correction: only a node-type argument is added; the type confusion is in five siblings.
9862	libexif	Size arithmetic may overflow before a bounds check.	Label correction: the wraparound is already blocked by the pre-fix size guard (pentax.c:394).
467	gnome-screensaver	Dialog exit during shaking may cause a race.	Label correction: CVE-2010-0732 lies in GTK+'s gdk/gdkwindow.c.

S5 Evidence cases and failure analysis

The following cases show how source identity, execution evidence, and oracle interpretation shape an audit decision. Each is drawn from retained experiments or review findings and keeps its original outcome provenance.

S5.1 A bounded negative with an explicit rejection rationale

BigVul instance 187491 labels Android's `sock_send_all`. The patch wraps a call to `send` in `TEMP_FAILURE_RETRY`. The pre-patch function already retries that call when it returns a negative value with `errno == EINTR`:

Listing 1. Existing retry logic in BigVul 187491.

```
do ret = send(sock_fd, buf, s, 0);
while (ret < 0 && errno == EINTR);
```

The saved attack interrupts a pending real system call using bounded `SIGALRM` delivery with `SA_RESTART` disabled. This exercises the condition that the new wrapper is intended to handle. On each source side, the logs record ten `EINTR` returns, eleven calls to `send`, successful delivery of all 512 bytes, and process exit code zero. Both runs have clean ASan and UBSan output. The debugger records eleven hits at the changed call on each side. Source-comparison records bind both tested functions to the complete upstream bodies at the verified parent and fixing revisions.

The behavioral oracle asks whether delivery completes after interruption, and the record therefore keeps the observed retries and the delivered byte count. The rejection rationale is that the existing loop already handles the exercised interruption condition, so the added wrapper is redundant for that behavior. The combination of this source explanation and the measured retry behavior supports rejecting the claimed defect under the saved condition. Acceptance requires the measured interruption, executed retry logic, and completed delivery together. The result is bounded by that condition and the saved calling context.

The same package contains a recorded cold replay. Its log reports 447 generated files before removal and zero afterwards, followed by successful build, attack, and disputed-line checks. It illustrates the S5 cold-replay procedure on a concrete package.

S5.2 Attribution across allocation and fault sites

BigVul instance 179087 concerns FFmpeg's `g2m_init_buffers`. A saved G2M4 input represents a 17×17 frame with 16×16 tiles. On the pre-patch side, the allocation is too small for the edge tile: ASan reports a one-byte write immediately beyond a 2,048-byte buffer. The invalid access occurs in `yuv2rgb`, while the allocation trace identifies `g2m_init_buffers`. The recorded exit codes are 1 before the patch and 0 after it. The controlled build comparison changes the relevant buffer-allocation hunk within the retained historical configuration.

The causal explanation includes both the allocation and the later access. The labelled function supplies the undersized allocation, and a downstream operation performs the invalid write. This supports attributing the defect to the allocator when the source change, allocation trace, and same-input comparison are considered together. Requiring the labelled function to be the top fault frame would miss this relationship; assigning the defect to every caller would also overstate the evidence.

S5.3 Failures of source and execution attribution

A compilable but incomplete function. BigVul 177956 contains a 23-line extract of a 129-line upstream function, and the omitted region contains the invalid access. The truncated body compiles

and returns cleanly, so its successful execution characterizes the extract alone; a judgment about the historical function requires its complete source. Source completeness must therefore be established before a clean attack result is interpreted.

Function entry with an uncovered disputed branch. In BigVul 180307 and 180308, the labelled functions were entered, while the branches affected by the patch recorded zero hits. Testing the proposed mechanism requires execution of those branches. A zero debugger hit count must also be checked for inlining before it is interpreted as nonexecution.

A shared file with multiple functions. For BigVul 180472, the labelled function is `ipow`, while the observed frames belong to `print_object`. Both occur in a 1,033-line translation unit containing 41 functions. Matching only the extracted filename incorrectly attributes those frames to the target. Qualified function identity and source-line ranges distinguish the two.

A caller fails before the target receives the input. The retained evidence for PrimeVul 199823 attributes the fault to the caller `HTMLlineproc2body`, which fails before the disputed input reaches the labelled function, `formList_addInput`. The observation is attributed to the caller; target confirmation requires its own causal execution evidence. Evidence linking an action of the labelled function to the affected object and to a violation in a callee would support a different attribution.

A label correction contradicted by a defect witness. DiverseVul instance 3313_7cca7e4c0083 was assigned to label correction, but its saved TGA attack produces a UBSan signed-overflow report in `fill_in_context` on both sides. The pre-patch report points to `io-tga.c:388`, within the labelled function's line range; the comparison-side report points to the corresponding operation at line 387. The pre-patch observation is a defect witness and contradicts the negative interpretation of the tested mechanism.

S5.4 The worked example in full: BigVul 182843–182846

This subsection expands the worked example of the main paper's method section with the retained source, attack history, and replay records.

S1: source identity and environment. The listed commit 3e5cdf1 has the single parent 3a90229, zstd 1.3.7; both full hashes appear in the main paper's citation of the commit. The labelled function occupies lines 2001–2041 of the parent's `lib/compress/zstd_compress.c` and lines 2001–2043 of the fixed file. The dataset's vulnerable body has 41 lines. Compared byte for byte with the parent's text, ten lines differ, each by one leading space introduced by extraction; when leading whitespace is ignored, all 41 lines match. This relationship is recorded in the sample, so later readings can distinguish the executed source from the dataset representation.

Of the 40 files in the retained tree, 39 are byte-identical to upstream at the parent by SHA-256. The 40th, `zstd_compress.c`, has upstream's own definition of `ZSTD_buildCTable` replaced by an include of `arm_body.inc`, into which the replay script copies either stored body; the removed upstream text is preserved in `splice.diff`. It matches the dataset's vulnerable body on all 41 lines when leading whitespace is ignored, and byte for byte on 31 of them. The stored bodies omit the return-type line, so the spliced file supplies `MEM_STATIC size_t` immediately before the include. The two body files that the replay script copies are byte-for-byte copies of the dataset's vulnerable and fixed bodies, respectively; neither is edited. The whole library is compiled twice per run with `gcc -O1 -g -fno-omit-frame-pointer -fsanitize=address`.

S2: the disputed operation. The commit's hunk in the labelled function is four lines:

Listing 2. The functional hunk of 3e5cdf1; the other three hunks in this file add `DEBUGLOG` calls only.

```

736     switch (type) {
737     case set_rle:
738         *op = codeTable[0];
739         CHECK_F(FSE_buildCTable_rle(nextCTable, (BYTE)max));
740         if (dstCapacity==0) return ERROR(dstSize_tooSmall);
741         *op = codeTable[0];
742         return 1;
743

```

ZSTD_compressSequences_internal builds the literal-length, offset and match-length tables from one output cursor, and the only capacity test it makes is $(oend-op) < 3+1$ before the sequences header. Whatever the earlier tables consume can exhaust the remaining capacity, so a later table can be entered with `dstCapacity == 0` and the `set_rle` store writes one byte past the caller's buffer.

S3: the three attack rounds. The first round (2026-07-05) compiled a harness whose own body performed the store. ASan reported a heap-buffer-overflow, but the summary line names the harness file, `182843.harness.c:10`. The executed store belongs to the harness. The superseded harness and log remain as an oracle-control record; function confirmation uses the source-bound attack below.

The second round (2026-08-01, 02:54) compiled `.search/search.c`, 44 lines adapted from upstream's own "too small destination buffer" fuzzer case. For compression levels 1 to 19 and 40 input patterns it compresses once to learn the output size, then for every truncated capacity below that size writes a four-byte end mark immediately past the buffer and checks whether ZSTD_compressCctx has altered it, saving the input and parameters on the first alteration. Its purpose was to locate a capacity at which the compressed offset table ends exactly on the buffer's last byte.

The third round (the same day, 11:30–11:42) fixed the input and wrote the two-side driver. The input is 40 records of four non-repeating bytes followed by the same 16-byte pattern. At level 1 this makes the match lengths equal, so upstream's ZSTD_selectEncodingType chooses `set_rle` for the match-length table while the offset codes vary and the offset table is `set_compressed`; the compressed offset table then consumes the remaining budget. The capacity is 192, the value the scan identified, and the driver reads it from `argv[1]` so the scan can be repeated at other capacities. The destination is allocated at exactly that size, so a one-byte overrun is visible. The driver supplies the requested capacity directly; the two stored source bodies provide the repair logic.

S4: the observed comparison. The two binaries hash differently. The vulnerable side exits 1; the fixed side exits 0 and prints ZSTD_compress returned error: Destination buffer is too small. The complete report follows:

Listing 3. Complete retained ASan report for BigVul 182843. Case-local workspace prefixes are shortened; the artifact preserves the original log.

```

775 [182843] ZSTD_compress(dst, dstCapacity=192, src, 800, level=1) [40 records of 4 literals + 16-
776   byte pattern]
777 =====
778 ==976883==ERROR: AddressSanitizer: heap-buffer-overflow on address 0x61000000100 at pc 0
779   x55fcb5f813cb bp 0x7fff6afdaba0 sp 0x7fff6afdab90
780 WRITE of size 1 at 0x61000000100 thread T0
781   #0 0x55fcb5f813ca in ZSTD_buildCTable arm_body.inc:14
782   #1 0x55fcb5f881f3 in ZSTD_compressSequences_internal upstream_zstd/lib/compress/zstd_compress.
783   c:2264
784   #2 0x55fcb5f881f3 in ZSTD_compressSequences upstream_zstd/lib/compress/zstd_compress.c:2315

```

```

785 #3 0x55fcb5f881f3 in ZSTD_compressBlock_internal upstream_zstd/lib/compress/zstd_compress.c
786 :2469
787 #4 0x55fcb5f8b55b in ZSTD_compress_frameChunk upstream_zstd/lib/compress/zstd_compress.c:2542
788 #5 0x55fcb5f8b55b in ZSTD_compressContinue_internal upstream_zstd/lib/compress/zstd_compress.c
789 :2691
790 #6 0x55fcb5f902e9 in ZSTD_compressEnd upstream_zstd/lib/compress/zstd_compress.c:3061
791 #7 0x55fcb5f9071e in ZSTD_compress_advanced_internal upstream_zstd/lib/compress/zstd_compress.
792 c:3123
793 #8 0x55fcb5f98193 in ZSTD_compress_usingDict upstream_zstd/lib/compress/zstd_compress.c:3135
794 #9 0x55fcb5f982a8 in ZSTD_compressCCtx upstream_zstd/lib/compress/zstd_compress.c:3145
795 #10 0x55fcb5f983d8 in ZSTD_compress upstream_zstd/lib/compress/zstd_compress.c:3155
796 #11 0x55fcb5f6c7c8 in main driver.c:75
797 #12 0x7f449a63fd8f in __libc_start_call_main ../sysdeps/nptl/libc_start_call_main.h:58
798 #13 0x7f449a63fe3f in __libc_start_main_impl ../csu/libc-start.c:392
799 #14 0x55fcb5f6c564 in _start (arm_vuln+0x3564)
800
801 0x61000000100 is located 0 bytes to the right of 192-byte region [0x610000000040,0x610000000100)
802 allocated by thread T0 here:
803 #0 0x7f449a8f3887 in __interceptor_malloc ../../../../src/libsanitizer/asan/asan_malloc_linux.
804 cpp:145
805 #1 0x55fcb5f6c761 in main driver.c:69
806 #2 0x7f449a63fd8f in __libc_start_call_main ../sysdeps/nptl/libc_start_call_main.h:58
807
808 SUMMARY: AddressSanitizer: heap-buffer-overflow arm_body.inc:14 in ZSTD_buildCTable
809 Shadow bytes around the buggy address:
810 0x0c207fff7fd0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
811 0x0c207fff7fe0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
812 0x0c207fff7ff0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
813 0x0c207fff8000: fa fa fa fa fa fa fa fa fa 00 00 00 00 00 00 00
814 0x0c207fff8010: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
815 =>0x0c207fff8020:[fa]fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
816 0x0c207fff8030: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
817 0x0c207fff8040: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
818 0x0c207fff8050: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
819 0x0c207fff8060: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
820 0x0c207fff8070: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
821 Shadow byte legend (one shadow byte represents 8 application bytes):
822 Addressable: 00
823 Partially addressable: 01 02 03 04 05 06 07
824 Heap left redzone: fa
825 Freed heap region: fd
826 Stack left redzone: f1
827 Stack mid redzone: f2
828 Stack right redzone: f3
829 Stack after return: f5
830 Stack use after scope: f8
831 Global redzone: f9
832 Global init order: f6
833 Poisoned by user: f7
834 Container overflow: fc
835 Array cookie: ac
836 Intra object redzone: bb
837 ASan internal: fe
838 Left alloca redzone: ca
839 Right alloca redzone: cb
840 Shadow gap: cc
841 ==976883==ABORTING

```

Line 14 of the spliced body is `*op = codeTable[0];`, the disputed store. The allocation trace points to the driver's own `malloc`, so the caller owns the overflowed output buffer. The labelled function performs the invalid store, so the defect is attributed to it. The trace also identifies the executed compression callers. Sibling-specific source bindings and execution observations are described below.

S5: package and replay. `EVIDENCE_MAP.json` lists each active file's relative path, purpose, size and SHA-256 digest, including the retained upstream tree, both stored bodies, the driver, the splice diff, both logs, the search program and both built binaries. The replay script rebuilds both sides from source, runs them under a 60-second bound, prints the comparison, and classifies the outcome as a repair differential, faults on both sides, inverted polarity, or clean oracle output on both sides, according to the actual observations.

The instance is also released as the case image `vulvalidate/bigvul-182843:1`, built on the base image `vulvalidate/repro-base:1`, whose Dockerfile pins Ubuntu 22.04 by digest. The build record reads the verdict from the sample's structured record and lists the replay script as the entry point, with the stored fixed body compiled as the comparison side. The replay command is

```
docker run --rm --network none vulvalidate/bigvul-182843:1
```

and Listing 4 reproduces the complete retained transcript of that fresh-container run. It reports the same exit codes 1 and 0, the same frame #0 at `arm_body.inc:14`, the same call path through frame #9, and the same `dstSize_tooSmall` return as the original record. The process identifier and the heap address differ between the two runs, as address space layout randomization implies; the reported source location and repair behavior agree. The image prints the observations and leaves their comparison with the recorded values to the reader.

Listing 4. Complete transcript of the fresh-container replay of BigVul 182843, retained as `docker_repro/runs/182843.log`. The script prints the first ten frames of the vulnerable-side log; the full log is in the package.

```
===== DIFFERENTIAL =====
oracle : AddressSanitizer (1-byte heap-buffer-overflow WRITE)
trigger : ZSTD_compress(dst, dstCapacity=192, src, 800, level=1) on
         40 records of 4 literals + a fixed 16-byte pattern
vuln arm rc : 1
fix arm rc : 0
--- vuln.asan.log (head) ---
[182843] ZSTD_compress(dst, dstCapacity=192, src, 800, level=1) [40 records of 4 literals + 16-byte pattern]
=====
==95==ERROR: AddressSanitizer: heap-buffer-overflow on address 0x51000000100 at pc 0x5565087643cb bp 0
x7fffd0f875c0 sp 0x7fffd0f875b0
WRITE of size 1 at 0x51000000100 thread T0
#0 0x5565087643ca in ZSTD_buildCTable arm_body.inc:14
#1 0x55650876b1f3 in ZSTD_compressSequences_internal upstream_zstd/lib/compress/zstd_compress.c:2264
#2 0x55650876b1f3 in ZSTD_compressSequences_internal upstream_zstd/lib/compress/zstd_compress.c:2315
#3 0x55650876b1f3 in ZSTD_compressBlock_internal upstream_zstd/lib/compress/zstd_compress.c:2469
#4 0x55650876e55b in ZSTD_compress_frameChunk_internal upstream_zstd/lib/compress/zstd_compress.c:2542
#5 0x55650876e55b in ZSTD_compressContinue_internal upstream_zstd/lib/compress/zstd_compress.c:2691
#6 0x5565087732e9 in ZSTD_compressEnd upstream_zstd/lib/compress/zstd_compress.c:3061
#7 0x55650877371e in ZSTD_compress_advanced_internal upstream_zstd/lib/compress/zstd_compress.c:3123
#8 0x55650877b193 in ZSTD_compress_usingDict upstream_zstd/lib/compress/zstd_compress.c:3135
#9 0x55650877b2a8 in ZSTD_compressCCtx upstream_zstd/lib/compress/zstd_compress.c:3145
--- fix.clean.log ---
[182843] ZSTD_compress(dst, dstCapacity=192, src, 800, level=1) [40 records of 4 literals + 16-byte pattern]
[182843] ZSTD_compress returned error: Destination buffer is too small
=====
RESULT: DIFFERENTIAL (vuln arm writes 1 byte past dst; fix arm returns dstSize_tooSmall)
```

S6: adjudication. Instance 182843 is *Confirmed*. The record keeps five qualifications. (i) The exercised mechanism is a missing output-capacity check, whereas the CWE-362 annotation and the advisory use race-condition terminology; the confirmed defect is the one that the commit’s hunk in this function removes, and the terminological disagreement is kept. (ii) The retained tree has one modified file, whose exact diff is preserved. (iii) The stored bodies begin at the function-name line, and the spliced file supplies the return type. (iv) The capacity of 192 bytes is specific to this input and compression level. (v) The earlier evidence was the single-side harness, and the current driver and replay script were written and run for this record.

The three siblings are label corrections based on a dynamic attack that produced no target-attributed defect. Their seven checks read as follows. Measured execution: exit code 0 on both sides with complete logs retained. Function execution: cold gcov reports `ZSTD_compressSequences_internal` called 12 times per side, `ZSTD_encodeSequences` 3 times per side, and the macro body of `ZSTD_encodeSequences` body materialized as `ZSTD_encodeSequences_default` and called 3 times per side. Attack input: real `ZSTD_compressCctx` and `ZSTD_compress_usingDict` calls and the `zstd` command line over repetitive 64 KiB, structured 98 KiB, deterministic 128 KiB, a 4 KiB dictionary and empty input, at destination capacities from 0 to 256, plus upstream’s own tests/fuzzer in a deterministic 200-case configuration on both sides. Oracle relevance: ASan, UBSan and LSan together with explicit prefix and suffix canaries around the destination, `zstd`’s own error behavior, the integer exit status, and a timeout; the canaries address the alleged mechanism directly, since an undersized-output write would alter them. Search scope: the entry points, inputs, destination capacities, and fuzzer cases listed above. Source completeness: each sibling’s stored bodies are token-equivalent to the complete upstream function at both revisions, with first and last lines and brace balance checked; Table S6 gives the parent and fixed line ranges (146 and 146 lines for 182844, 23 and 24 for 182845, and 86 and 90 for 182846). Patch execution: the disputed site executed 3 times per side for 182844 (parent line 2293, fixed line 2300), 3 times per side for 182845, and 3 and 4,758 times for the two changed lines of 182846.

Table S6 collects the per-sibling evidence. The three siblings share one whole-library attack campaign: a single instrumented build of the library contains all three functions, so one gcov report per side covers them together. Each sibling directory holds a copy of that campaign’s script, inputs, logs, and gcov reports, alongside its own stored bodies, source-completeness check, record, and cold-replay return code.

The rejection rationale is source-grounded: the commit’s only behavioral guard is in the sibling function, and a `DEBUGLOG` statement expands to an empty statement at the default debug level. The recorded rationale therefore concerns the advisory’s attribution under the exercised paths. Each sibling carries a recorded cold replay returning 0. Target-only ablation remains outside the measured scope: it requires a positive full-project differential to isolate a causal signal. The attack history retains the source explanation, observed behavior, and this scope limit.

S5.5 A resource limit changes the oracle’s meaning

BigVul 183359 tests `libpcap`’s `pcap_ng_check_header` with a 32-byte file declaring a 10 MiB section-header block. The experiment sets ASan’s maximum allocation size to 4 MiB. The pre-patch execution requests 10 MiB and terminates with an allocation-size report; the comparison side rejects the declared block length before that allocation. Both sides accept the valid control input.

The pre-patch code already enforces a 16 MiB bound, and the patch tightens the relevant bound to 1 MiB. The observed event is therefore an allocation-limit violation under the configured policy, and the configured 4 MiB limit is part of the experiment’s interpretation. A deployment-level denial-of-service claim additionally requires a justified resource model and evidence that an attacker can create the corresponding resource pressure.

Table S6. Per-instance evidence for the four functions labelled by commit 3e5cdf1. Calls and disputed-line executions are gcov counts per side under the attack inputs. Upstream lines give the complete function range to which the stored body is token-equivalent at the parent and fixing revisions. Return codes are the vulnerable and fixed sides, and the last column gives the recorded cold-replay return code.

ID	Labelled function	Calls	Upstream lines (parent; fixed)	Disputed lines executed	Codes	Cold
182843	ZSTD_buildCTable	–	–	set_rle store (body line 14), ASan WRITE on the vulnerable side	1/0	1/0
182844	ZSTD_compressSequences_internal	12	2191–2336; 2198–2343	added DEBUGLOG site, 3 per side (parent 2293, fixed 2300)	0/0	0
182845	ZSTD_encodeSequences	3	2166–2188; 2172–2195	inserted DEBUGLOG (fixed line 2179) and its parent counterpart, 3 per side	0/0	0
182846	ZSTD_encodeSequences_body (as _default)	3	2044–2129; 2046–2135	fixed lines 2059 and 2121, 3 and 4,758 times, with the corresponding parent lines	0/0	0

S6 Vulnerability detectors

This section documents the five detectors reported in the main paper: CodeT5, LineVul, DeepDFA, Devign, and ReVeal. We distinguish the BigVul and DiverseVul label-comparison experiments from the merged-benchmark experiment because their inputs, training runs, and evaluation populations differ. Each corrected-training result describes an individual run; comparisons across label settings are descriptive.

S6.1 Vulnerability detectors: background and implementation

The study evaluates five learning-based detectors that are widely used in function-level vulnerability detection: two pretrained transformers, CodeT5 and LineVul, and three graph-based models, DeepDFA, Devign, and ReVeal. Benchmark practice in this area concentrates on a small number of datasets: a survey of 81 vulnerability-detection papers found that 53 used at least one of BigVul, Devign, or ReVeal [24]. Each subsection below summarizes how a detector represents a function, where it has been trained and tested in prior vulnerability-detection work, and how this study implements it. In this study, every detector is trained and tested on the original labels of BigVul, DiverseVul, and PrimeVul (*O/O*) and on the corrected labels of each dataset (the main paper’s detector table and Table S7).

S6.1.1 CodeT5. CodeT5 is an identifier-aware encoder–decoder model pretrained on source code and natural language; its original evaluation includes defect detection, framed as binary classification of C functions [30]. In vulnerability detection, CodeT5 serves as a representative code language model. The DiverseVul study fine-tunes CodeT5 on DiverseVul together with earlier datasets [6]. The PrimeVul study fine-tunes code language models, including CodeT5, on PrimeVul and contrasts their scores with those obtained on earlier benchmarks such as BigVul, finding much lower performance under PrimeVul’s labelling and splits [9].

The implementation represents a function using the decoder hidden state at the end-of-sequence token and maps this representation to two class logits.

S6.1.2 LineVul. LineVul fine-tunes the encoder-only CodeBERT model on function tokens for function-level vulnerability prediction and uses self-attention scores to rank lines within predicted vulnerable functions [11, 13]. Its original evaluation trains and tests on BigVul and compares against graph-based baselines, including Devign and ReVeal, on the same data [13]. An empirical study of deep-learning vulnerability detectors evaluates LineVul alongside Devign and ReVeal on the Devign and BigVul datasets [28], and DeepDFA reports improvements when its dataflow embedding is combined with LineVul [27].

The BigVul, PrimeVul, and merged text experiments use sequences of at most 512 tokens, and longer functions are truncated. The BigVul and merged experiments use LineVul’s fixed threshold of 0.5; the PrimeVul experiment uses validation-selected thresholds, as described in Section S6.4.

S6.1.3 DeepDFA. DeepDFA embeds abstract dataflow facts, derived from definitions and their API calls, data types, literals, and operators, into a gated graph neural network over control-flow graphs, following the structure of a reaching definitions analysis [27]. Its original evaluation trains and tests on BigVul, reports efficient training relative to transformer detectors, and combines DeepDFA with LineVul to improve detection on the same dataset [27].

The implementation uses control-flow graphs derived with Joern [31] and the abstract-dataflow node features of the original pipeline. Graph construction restricts each dataset to functions with a buildable graph, so DeepDFA’s test population is a subset of the text population.

S6.1.4 Devign. Devign builds a composite program graph from syntax, control-flow, data-flow, and natural-code-sequence edges, encodes it with a gated graph neural network, and classifies the resulting representation through a convolutional readout [32]. Its dataset, collected from open-source C projects and released for FFMpeg and QEMU, became a standard vulnerability-detection benchmark, and Devign is a common graph-based baseline: LineVul compares against it on BigVul [13], and the empirical study above evaluates it on the Devign and BigVul datasets [28].

S6.1.5 ReVeal. ReVeal first learns graph representations of functions with a gated graph neural network, then rebalances the classes and trains a classifier with a metric-learning objective that separates vulnerable from non-vulnerable representations [5]. Its original evaluation uses a dataset collected from Chromium and Debian, together with the FFMpeg and QEMU data of Devign [5]. ReVeal represents graph neural networks in the DiverseVul study, which trains and tests it on DiverseVul and earlier datasets [6]; LineVul compares against it on BigVul [13]; and the empirical study above includes it on the Devign and BigVul datasets [28].

S6.1.6 Shared graph inputs for Devign and ReVeal. In the merged experiment, Devign and ReVeal share Joern-derived [31] graphs with syntax, control-dependence, control-flow, and reaching-definition edges, including reverse edges. Their node features combine a node-type identifier with an embedding of the node’s code. Graph construction and filtering are part of the evaluated pipeline.

S6.2 Label-comparison experiments

The notation O/O , C/C , and O/C follows the main paper. The first letter specifies the training labels and the second the evaluation labels. Within each matched run, O/C reuses the original checkpoint and prediction scores. Accuracy, precision, recall, F1, MCC, and AUROC use the same prediction population and label vector for each reported row. Threshold selection follows the protocol in the main paper.

BigVul training and run selection. For CodeT5, the original and corrected runs both use unweighted loss. DeepDFA uses hidden dimension 128 and five message-passing steps in both runs; checkpoint selection monitors validation F1 after threshold selection. The corrected Devign run and ReVeal’s

graph stage use 171 input features, graph embedding dimension 200, and six message-passing steps. AUROC is computed from the scores and labels used for the classification metrics, giving 0.6954, 0.7049, and 0.7326 for Devign’s O/O , O/C , and C/C rows, respectively. ReVeal’s corresponding values are 0.7294, 0.7135, and 0.6486. These populations and label vectors also define the E analysis below.

DiverseVul comparison scope. The split groups functions by whitespace-normalized body so that one group belongs to one split. Corrected test functions form a subset of the original test functions, disjoint from the original training and validation sets.

The retained CodeT5 experiment uses unweighted classification. Devign and ReVeal share a graph baseline with hidden dimension 256. The implementation uses learned token embeddings and class-weighted cross-entropy with weights $[1, 10]$, and caps graphs at 500 nodes and 10,000 edges. DeepDFA uses balanced undersampling, unweighted loss, and validation F1 after threshold selection.

Corrected training omits unresolved instances. The O/C – C/C comparison therefore measures the combined effect of retraining and the data changes.

S6.3 Common-test-set evaluation: setting E

Setting E starts from the corrected test population and removes label-correction instances, retaining the other instances and their corrected labels. Confirmed instances retain $y = 1$, retained original non-vulnerable instances retain $y = 0$, and unresolved outcomes remain outside the learning population. Original-trained and corrected-trained checkpoints are evaluated on the same retained functions for each detector.

The BigVul score-level implementation removes label-correction instances carrying $y = 0$ in the detector’s frozen corrected label vector. The artifact preserves the removal lists.

For DiverseVul, the correction instances are removed before model-specific graph filtering. Each checkpoint retains the operating threshold used on the corrected test set. Thus O/E and C/E change the tested population while preserving the learned scores and threshold policies.

For a checkpoint, removal of negatives preserves true positives, false negatives, and recall. False positives can decrease, so precision and F1 can increase through test-set filtering alone. The comparison of O/E with C/E describes the training regimes on a common retained population; comparisons of a corrected-test row with its E row describe the effect of that filtering. The main paper’s BigVul E results illustrate their distinct roles: DeepDFA’s corrected-training F1 exceeds original-training F1 by 0.0173, while CodeT5 and LineVul retain higher F1 under original training on E .

S6.4 PrimeVul detector performance

Table S7 reports PrimeVul results in the five settings of the main paper’s detector table. Following Section S6.3, E removes the label-correction instances carrying $y = 0$ in the corrected labels.

Thresholds maximize validation F1 and are frozen before test evaluation. In O/O and C/C the validation labels match the training labels; in O/C , DeepDFA, Devign, ReVeal, and LineVul reselect their thresholds on corrected validation labels, whereas CodeT5 retains its original threshold. Both E settings retain the threshold of the corresponding corrected-test row. LineVul uses validation-selected thresholds here rather than 0.5 because the corrected-trained LineVul predicts no positive instance at 0.5.

Each graph pipeline evaluates its graph-supported subset of the text population, and each E population omits the removed instances.

Table S7. Vulnerability detection performance on PrimeVul. Each model has five settings, O/O , C/C , O/C , O/E , and C/E , where E removes only label-correction instances from the corrected test set and leaves the remaining instances and labels unchanged. Graph availability restricts model-specific test populations. Thresholds are selected on validation data as described in the text. Leading zeros are omitted.

Model	Train	Test	PrimeVul					
			ACC	Prec.	Rec.	F1	MCC	AUROC
CodeT5	O	O	.9434	.1459	.3206	.2006	.1902	.8489
	C	C	.9359	.1413	.3545	.2020	.1954	.8443
	O	C	.9428	.1476	.3139	.2008	.1887	.8466
	O	E	.9429	.1477	.3139	.2009	.1888	.8466
	C	E	.9360	.1414	.3545	.2022	.1956	.8444
LineVul	O	O	.9584	.1810	.2495	.2098	.1916	.8127
	C	C	.9472	.1487	.2769	.1935	.1775	.8361
	O	C	.9566	.1802	.2533	.2106	.1918	.8090
	O	E	.9566	.1803	.2533	.2106	.1918	.8091
	C	E	.9472	.1487	.2769	.1935	.1775	.8361
DeepDFA	O	O	.4418	.0310	.7996	.0597	.0694	.6927
	C	C	.1936	.0271	.9824	.0528	.0625	.6933
	O	C	.4421	.0319	.7972	.0614	.0698	.6890
	O	E	.4421	.0319	.7972	.0614	.0698	.6890
	C	E	.1936	.0271	.9824	.0528	.0625	.6933
DeSign	O	O	.9380	.1158	.2746	.1629	.1501	.7426
	C	C	.9392	.1359	.3132	.1896	.1786	.7738
	O	C	.9376	.1188	.2726	.1654	.1512	.7403
	O	E	.9376	.1189	.2726	.1655	.1513	.7403
	C	E	.9393	.1360	.3132	.1897	.1787	.7738
ReVeal	O	O	.9470	.1279	.2426	.1675	.1507	.7593
	C	C	.9404	.1197	.2562	.1632	.1472	.7504
	O	C	.9466	.1318	.2420	.1706	.1528	.7576
	O	E	.9466	.1318	.2420	.1706	.1527	.7576
	C	E	.9404	.1198	.2562	.1633	.1473	.7504

Overall changes (O/O to C/C). Unlike BigVul, where F1 decreases for all five detectors, PrimeVul F1 increases for CodeT5 and DeSign and decreases for LineVul, DeepDFA, and ReVeal. DeSign shows the largest change, from 0.1629 to 0.1896. As in the main paper, these comparisons combine changes in training data, evaluation labels, and retained populations.

Re-evaluating models without retraining (O/O to O/C). The main paper’s pattern does not recur. F1 does not decrease for any detector; it increases by 0.0002 to 0.0031, while AUROC decreases slightly for all five detectors, by 0.0017 to 0.0037. Precision increases and recall decreases for CodeT5, DeepDFA, DeSign, and ReVeal. Reselection on corrected validation labels changes the threshold only for LineVul; for every other detector the O/C threshold equals the O/O threshold.

Retraining with corrected labels (O/C to C/C). F1 improves for CodeT5 and DeSign and decreases for LineVul, DeepDFA, and ReVeal, compared with four of five detectors improving on each main-paper dataset. MCC improves for CodeT5 and DeSign. Recall increases for all five detectors (0.0142

to 0.1852), so the corrected-trained checkpoints operate at higher-recall points. Higher AUROC can coexist with lower F1: LineVul’s AUROC rises from 0.8090 to 0.8361 while its F1 falls by 0.0171.

Withholding the relabelled instances (O/E versus C/E). For each checkpoint, removing the correction-derived negatives at a fixed threshold leaves recall unchanged and changes no metric by more than 0.0003. The corrected-training difference keeps its sign on the common test view for every detector: C/E F1 exceeds O/E for CodeT5 and Devign and falls below it for LineVul, DeepDFA, and ReVeal. Unlike BigVul, no sign changes between C and E, because PrimeVul’s E removes few test instances.

Implications for RQ4. The main paper’s finding that corrected evaluation lowers F1 for every detector concerns BigVul and DiverseVul. On PrimeVul, corrected evaluation does not lower F1, and retraining with corrected labels yields small, detector-dependent changes rather than the consistent gains observed on BigVul and DiverseVul. This agrees with the audit outcome: most audited PrimeVul positives are confirmed. Accuracy above 0.93 for four detectors reflects the non-vulnerable majority; apart from DeepDFA, no detector finds more than 37% of the vulnerable functions, and DeepDFA reaches recall of at least 0.79 with precision below 0.033 in every setting.

S6.5 Merged-benchmark training and selection

DeepDFA, ReVeal, and Devign evaluate the functions for which their graph preprocessing succeeds, whereas CodeT5 and LineVul use the full text input population. Graph filtering therefore changes the evaluation population even when the underlying split assignment is shared.

DeepDFA uses hidden dimension 32, five message-passing steps, attention pooling, and three output layers. All four abstract-dataflow feature sets are concatenated, and the node-loss weight is 0.3. It uses the recorded undersampling configuration and unweighted loss. ReVeal’s graph encoder has dimension 200 and six message-passing steps. Its metric stage uses hidden dimension 400, internal dimension 200, dropout 0.2, margin 0.5, and loss weights 0.5 and 0.001. The retained metric-stage run skipped one batch with a non-finite loss or gradient and reused the preceding graph-stage checkpoint. Devign uses binary cross-entropy, random sampling, and equal positive and negative loss weights.

For CodeT5, checkpoint selection uses validation F1 at threshold 0.5; after loading the selected checkpoint, the threshold is selected on validation and frozen for testing. The final thresholds are 0.2816 for CodeT5, 0.5 for LineVul, 0.7234 for DeepDFA, 0.2697 for Devign, and 0.1401 for ReVeal. The merged experiment compares the retained corrected-data detectors; a matched original-label training comparison remains a separate experiment.

S6.6 Metric computation

For the merged experiment, one implementation recomputes accuracy, precision, recall, F1, and MCC from saved labels and prediction scores. A function is predicted vulnerable when its score is at least the frozen threshold. Precision, recall, and F1 are set to zero when their denominator is zero. AUROC is computed from continuous scores. Threshold-free metrics still depend on the evaluated population and label mapping.

S7 Operational setup and cost

The campaign used concurrent Claude Code sessions on two Linux workstations, with each session supporting multiple agents. As reported in the main paper, the S1–S6 workflow cost approximately 20 minutes of agent time per instance on average, with API usage equivalent to approximately USD 60,000. These are campaign-level estimates covering source reconstruction, attack development and execution, evidence preservation, and adjudication. The subsequent two-expert review is a separate activity.

Table S8. Artifact components and the evidence they provide. A component’s presence enables inspection; its reconstruction status is established separately for each instance that uses it.

Component	Retained material and interpretation
Operational protocol	Skill, dataset-specific prompts, project brief, field definitions, and checker scripts document the required procedure.
Sample records	Stored bodies, metadata, structured decisions, and evidence references identify the audited function and the stated outcome.
Local evidence packages	Scripts, inputs, harnesses, complete logs, execution proof, and evidence maps support inspection and replay where the required files are present.
Shared builds	Historical source or pinned build trees, configuration, dependencies, and local patches identify reused execution environments.
Docker packages	Case Dockerfiles, base-image pins, dependency recipes, image identities, and replay commands bind container-compatible experiments to their execution environments. Fresh-container execution and cold rebuilding are checked separately.
Review materials	Sampling code, manifests, and eligibility and exclusion records document the selected review population. Individual expert assessments and final consensus verdicts determine eligibility for reviewed instances.
Model experiments	Run configurations, checkpoint-selection summaries, saved scores and labels, and metric scripts specify the reported evaluation populations.
Release statistics	Source manifests, split counts, metadata checks, and complete project/CWE CSVs document the released resource summarized in the main paper.

Historical source, dependencies, configurations, and compiler caches are reused within projects, and instances share a build only when the relevant revisions and tested code match. The protocol limits heavy builds to two or three concurrent jobs and compilation to eight parallel tasks per build. Executions use recorded resource limits and timeouts. These controls describe the operating procedure. Evaluating their effect on throughput requires a controlled comparison of concurrency settings.

The USD figure is the API-equivalent price of the aggregate model usage; subscription fees, researcher time, hardware, and electricity are accounted separately, and a per-dataset, per-stage, or per-outcome breakdown would require corresponding usage records. Concurrent agent time also differs from calendar time, and reconstructing a full project costs substantially more than replaying a prepared experiment.

S8 Artifact completeness and reproducibility

S8.1 Components and provenance

Table S8 summarizes the retained components and their evidence scopes. The datasets, evidence, replay materials, and Docker release are available through <https://anonymous.4open.science/r/VulnIValidate-3E68>. The operational skill is available through <https://anonymous.4open.science/r/vuln-label-dynamic-confirmation-skill-2849>. These are the anonymized entry points specified in the main paper. The repository copy under `tasks/skills/vuln-label-dynamic-confirmation/` includes the skill, dataset-specific references, and checker wrappers. The longer prompts preserve historical changes and dataset-specific rules; the English specification in Sections S1–S3 organizes the contract used in the main paper.

Local source bodies bind an extracted experiment to its target. Shared build pins identify a historical project configuration but require the corresponding source, dependencies, and build steps for reconstruction. These two forms of availability have different reproduction requirements.

S8.2 Established checks and remaining limits

The main paper refers to this subsection for the gaps in replay and reproduction. The list below states each gap and the material that would close it; the paragraphs after the list summarize the checks already established.

- G1. *Prebuilt image components*. Some case images compile an upstream library or the tested executables during image construction. Fresh-container execution demonstrates isolation from the original workspace; cold source reconstruction of those components requires a record of the removed products and the rerun source and dependency steps.
- G2. *Platform-bound experiments*. Kernel, virtual-machine, device, and privilege-dependent experiments retain their requirements explicitly and replay on a matching platform outside the container release.
- G3. *Shared builds*. Shared build pins identify a historical configuration and require the corresponding source, dependencies, and build steps for reconstruction.

The released-data validation reports successful source-metadata linkage for all 537,591 rows and verifies byte equality of original and relabelled rows with their source functions using SHA-256. The concatenated train, validation, and test files match the merged release byte-for-byte. These checks establish data and metadata consistency. Dynamic reconstruction has its own build and replay status.

The anonymous repositories named above are the release entry points in the main paper. The operational skill documents the procedure; the evidence collection supplies its instance-specific inputs and outputs. Outcome, review, image, and learning-split manifests must refer to the same frozen identifiers. Later additions, reproductions, or revised decisions are kept separate from the materials used for the reported experiments.

References

- [1] Anthropic. 2026. Claude Code Documentation. <https://code.claude.com/docs/en/overview> Accessed 2026-09-17.
- [2] Anthropic. 2026. Introducing Claude Opus 5. <https://www.anthropic.com/news/claude-opus-5> Accessed 2026-09-17.
- [3] Earl T Barr, Mark Harman, Phil McMinn, Muzammil Shahbaz, and Shin Yoo. 2015. The oracle problem in software testing: A survey. *IEEE transactions on software engineering* 41, 5 (2015), 507–525. doi:10.1109/TSE.2014.2372785
- [4] Fabrice Bellard. 2005. QEMU, a fast and portable dynamic translator. In *2005 USENIX Annual Technical Conference (USENIX ATC 05)*. USENIX Association, 41–46. <https://www.usenix.org/conference/2005-usenix-annual-technical-conference/qemu-fast-and-portable-dynamic-translator>
- [5] Saikat Chakraborty, Rahul Krishna, Yangruibo Ding, and Baishakhi Ray. 2021. Deep learning based vulnerability detection: Are we there yet? *IEEE Transactions on Software Engineering* 48, 9 (2021), 3280–3296.
- [6] Yizheng Chen, Zhoujie Ding, Lamya Alowain, Xinyun Chen, and David Wagner. 2023. DiverseVul: A New Vulnerable Source Code Dataset for Deep Learning Based Vulnerability Detection. In *Proceedings of the 26th international symposium on research in attacks, intrusions and defenses*. 654–668.
- [7] Jacob Cohen. 1960. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement* 20, 1 (1960), 37–46.
- [8] Roland Croft, M Ali Babar, and M Mehdi Kholoosi. 2023. Data quality for software vulnerability datasets. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 121–133.
- [9] Yangruibo Ding, Yanjun Fu, Omniyyah Ibrahim, Chawin Sitawarin, Xinyun Chen, Basel Alomair, David Wagner, Baishakhi Ray, and Yizheng Chen. 2024. Vulnerability detection with code language models: How far are we? arXiv preprint arXiv:2403.18624.
- [10] Jiahao Fan, Yi Li, Shaohua Wang, and Tien N Nguyen. 2020. A C/C++ code vulnerability dataset with code changes and CVE summaries. In *Proceedings of the 17th international conference on mining software repositories*. 508–512.

- [11] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, et al. 2020. CodeBERT: A pre-trained model for programming and natural languages. In *Findings of the association for computational linguistics: EMNLP 2020*. 1536–1547.
- [12] Free Software Foundation. 2026. Debugging with GDB: The GNU Source-Level Debugger. GNU GDB manual. <https://sourceware.org/gdb/current/onlinedocs/gdb.html/> Accessed 2026-09-17.
- [13] Michael Fu and Chakkrit Tantithamthavorn. 2022. LineVul: A Transformer-based Line-Level Vulnerability Prediction. In *Proceedings of the 19th international conference on mining software repositories*. 608–620.
- [14] GNU Project. 2026. gcov: A Test Coverage Program. GCC documentation. <https://gcc.gnu.org/onlinedocs/gcc/Gcov.html> Accessed 2026-09-17.
- [15] Ahmad Hazimeh, Adrian Herrera, and Mathias Payer. 2020. Magma: A ground-truth fuzzing benchmark. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 4, 3 (2020), 1–29.
- [16] Avi Kivity, Yaniv Kamay, Dor Laor, Uri Lublin, and Anthony Liguori. 2007. kvm: the Linux virtual machine monitor. In *Proceedings of the Linux symposium*, Vol. 1. Ottawa, Ontario, Canada, 225–230. <https://www.kernel.org/doc/ols/2007/ols2007v1-pages-225-230.pdf>
- [17] Linux Kernel Documentation. 2026. Kernel Address Sanitizer. <https://docs.kernel.org/dev-tools/kasan.html> Accessed 2026-09-17.
- [18] Linux Kernel Documentation. 2026. Kernel Concurrency Sanitizer. <https://docs.kernel.org/dev-tools/kcsan.html> Accessed 2026-09-17.
- [19] Linux Kernel Documentation. 2026. Kernel Memory Sanitizer. <https://docs.kernel.org/dev-tools/kmsan.html> Accessed 2026-09-17.
- [20] LLVM Project. 2026. LeakSanitizer. Clang documentation. <https://clang.llvm.org/docs/LeakSanitizer.html> Accessed 2026-09-17.
- [21] LLVM Project. 2026. UndefinedBehaviorSanitizer. Clang documentation. <https://clang.llvm.org/docs/UndefinedBehaviorSanitizer.html> Accessed 2026-09-17.
- [22] Dirk Merkel. 2014. Docker: lightweight linux containers for consistent development and deployment. *Linux Journal* 239, 2 (2014), 2. <https://www.linuxjournal.com/content/docker-lightweight-linux-containers-consistent-development-and-deployment>
- [23] Nicholas Nethercote and Julian Seward. 2007. Valgrind: a framework for heavyweight dynamic binary instrumentation. *ACM Sigplan notices* 42, 6 (2007), 89–100.
- [24] Niklas Risse, Jing Liu, and Marcel Böhme. 2025. Top score on the wrong exam: On benchmarking in machine learning for vulnerability detection. *Proceedings of the ACM on Software Engineering* 2, ISSTA (2025), 388–410.
- [25] Konstantin Serebryany, Derek Bruening, Alexander Potapenko, and Dmitriy Vyukov. 2012. AddressSanitizer: A fast address sanity checker. In *2012 USENIX annual technical conference (USENIX ATC 12)*. 309–318.
- [26] Konstantin Serebryany and Timur Iskhodzhanov. 2009. ThreadSanitizer: data race detection in practice. In *Proceedings of the workshop on binary instrumentation and applications*. 62–71.
- [27] Benjamin Steenhoeck, Hongyang Gao, and Wei Le. 2024. Dataflow analysis-inspired deep learning for efficient vulnerability detection. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 1–13.
- [28] Benjamin Steenhoeck, Md Mahbubur Rahman, Richard Jiles, and Wei Le. 2023. An empirical study of deep learning models for vulnerability detection. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2237–2248.
- [29] Evgeniy Stepanov and Konstantin Serebryany. 2015. MemorySanitizer: fast detector of uninitialized memory use in C++. In *2015 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. IEEE, 46–55.
- [30] Yue Wang, Weishi Wang, Shafiq Joty, and Steven CH Hoi. 2021. CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. In *Proceedings of the 2021 conference on empirical methods in natural language processing*. 8696–8708.
- [31] Fabian Yamaguchi, Nico Golde, Daniel Arp, and Konrad Rieck. 2014. Modeling and discovering vulnerabilities with code property graphs. In *2014 IEEE symposium on security and privacy*. IEEE, 590–604.
- [32] Yaqin Zhou, Shangqing Liu, Jingkai Siow, Xiaoning Du, and Yang Liu. 2019. Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks. In *Advances in neural information processing systems*, Vol. 32.